

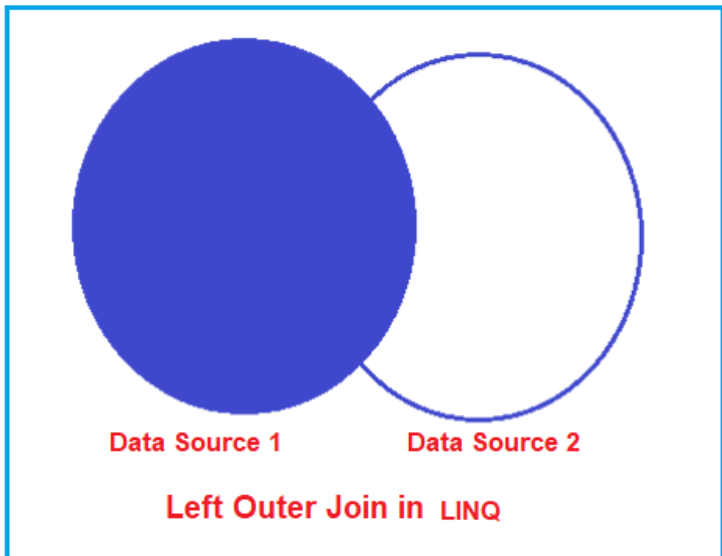
LINQ Left Outer Join in C# with Examples

In this article, I will discuss **LINQ Left Outer Join in C#** with Examples using Method and Query Syntax. The Left Join is also called the Left Outer Join. Please read the following three articles before proceeding to this article, as they all are required to understand the Left Outer Join.

1. [Inner Join in LINQ](#)
2. [Group Join in LINQ](#)
3. [Select Many In LINQ](#)

What is Left Join in SQL?

A Left Join, often referred to as a Left Outer Join, is a type of SQL join that combines rows from two or more tables based on a related column between them but prioritizes the first (left-most) table in the join. It returns all rows from the left table and the matched rows from the right table. Where there is no match, the result is NULL on the side of the right table. For a better understanding, please look at the following diagram, which shows the graphical representation of the Left Outer Join.



Here's how it works:

- **Matching Rows:** For each row in the left table, the Left Join looks for rows in the right table that match on the specified condition. If it finds one or more matching rows in the right table, it combines the columns of the left and right tables for each match found.
- **Non-matching Rows:** If no matching rows are found in the right table for a row in the left table, the query returns the columns from the left table, filling in NULLs for the right table.

The SQL syntax for a Left Join is as follows:

```
SELECT columns
FROM table1
LEFT JOIN table2
ON table1.common_column = table2.common_column;
```

Here,

- **columns:** The columns you want to retrieve.
- **table1:** The left table from which all rows will be returned.
- **table2:** The right table from which only matching rows will be returned.
- **common_column:** The column or condition matches rows between the two tables.

Left Joins are particularly useful when you want to find records in one table with no corresponding records in another table or when you need a complete list of records from the left table while still pulling in related data from another table when available.

Left Outer Join in LINQ

In LINQ (Language Integrated Query), a Left Join is a way to combine elements from two sequences (e.g., collections, arrays, lists) based on matching keys, where every element from the first (left) sequence is returned regardless of whether there is a matching element in the second (right) sequence. If there is no matching element in the right sequence, the result will include a default value for the type of the elements in the right sequence.

The Left Join in LINQ is performed using the join keyword, into keyword, and the DefaultIfEmpty() method call. This approach allows you to specify a join condition between two sources and then, for each element in the left source, select elements that have matching keys in the right source. When there's no match, the right side will have a default value, typically null for reference types or the default value for value types.

Examples to Understand Left Outer Join in LINQ:

Let us Understand Left Outer Join in LINQ with Examples using C# Language. We will use the following Employee and Address Data Sources to understand LINQ Left Outer Join. First, create a class file named **Employee.cs** and copy and paste the following code. This class has three properties: ID, Name, and AddressId. We have also created one method, GetAllEmployees(), which will return a collection of Employees, one of our data sources.

```
using System.Collections.Generic;
namespace LINQLeftOuterJoin
{
    public class Employee
    {
        public int ID { get; set; }
        public string Name { get; set; }
        public int AddressId { get; set; }
        public static List<Employee> GetAllEmployees()
        {
            return new List<Employee>()
            {
                new Employee { ID = 1, Name = "Preety", AddressId = 1},
                new Employee { ID = 2, Name = "Priyanka", AddressId = 2},
                new Employee { ID = 3, Name = "Anurag", AddressId = 0},
                new Employee { ID = 4, Name = "Pranaya", AddressId = 0},
                new Employee { ID = 5, Name = "Hina", AddressId = 5},
                new Employee { ID = 6, Name = "Sambit", AddressId = 6}
            }
        }
    }
}
```

```

    };
}
}
}

```

Next, create another class file with the name **Address.cs** and copy and paste the following code. This class has two properties, i.e., ID and AddressLine. We have also created one method, i.e., GetAddress(), which will return a collection of addresses, which will be our second data source.

```

using System.Collections.Generic;
namespace LINQLeftOuterJoin
{
    public class Address
    {
        public int ID { get; set; }
        public string AddressLine { get; set; }
        public static List<Address> GetAddress()
        {
            return new List<Address>()
            {
                new Address { ID = 1, AddressLine = "AddressLine1"},
                new Address { ID = 2, AddressLine = "AddressLine2"},
                new Address { ID = 5, AddressLine = "AddressLine5"},
                new Address { ID = 6, AddressLine = "AddressLine6"},
            };
        }
    }
}

```

In our example, we created two methods to return the respective Employees and Addresses, which will be our data sources. We have hard-coded the data sources, but you will get the data from a database in real-time. If you notice, we have two employees with address ID 0, which means these two employees do not have a matching address in the address data source.

Examples to Understand LINQ Left Outer Join using Query Syntax in C#:

Let us see an example of Understanding LINQ Left Outer Join using Query Syntax using C#. To perform the left outer join using query syntax in LINQ, we need to call the **DefaultIfEmpty()** method on the results of a Group Join. In LINQ Query Syntax, no such Group Join operator is available. We need to use the **LINQ Inner Join** and the **into** operator discussed in our [Previous Article](#). Let's see the step-by-step procedure to implement the LINQ Left Outer Join in C#.

Step 1:

The first step in implementing the LINQ Left Outer Join is to perform Group Join and store the result in a variable using the **"into"** operator. In the code snippet below, as you can see, the list of Employees is inner-joined with the list of Addresses based on the AddressId Property of the Employee object that matches the ID property of the Address object. And then, we store the result in the **EmployeeAddressGroup** using the **"into"** operator. The following code exactly does the same thing.

```

from emp in Employee.GetAllEmployees()
join add in Address.GetAddress()
on emp.AddressId equals add.ID
into EmployeeAddressGroup

```

Step 2:

In the second step, we need to include each element of the first (i.e., left) data source in the result set, irrespective of whether that element has any matching element found in the second (i.e., right) data source. To do this, we need to call the **DefaultIfEmpty()** method on each sequence of matching elements from the group join.

In our example, we need to use the **DefaultIfEmpty()** method on each sequence of matching Address objects. The **DefaultIfEmpty()** method returns a collection containing a default value if the sequence of matching Address objects is empty for any Employee object, ensuring that each Employee object is represented in the result collection. That means if the corresponding value is empty, the default value will be taken based on the data type. The following code exactly does the same thing.

```

from address in EmployeeAddressGroup.DefaultIfEmpty()

```

The default value for a reference type is null. So, you need to check for the null reference before accessing each element of the Address collection. The complete example code is given below. In the below example, we are implementing the LINQ Left Outer Join using Query Syntax. In the below example, we are projecting the result to an anonymous type.

```

using System.Linq;
using System;
namespace LINQLeftOuterJoin
{
    class Program
    {
        static void Main(string[] args)
        {
            //Performing Left Outer Join using LINQ using Query Syntax
            //Left Data Source: Employees
            //Right Data Source: Addresses
            //Note: Left and Right Data Source Matters
            var QSOuterJoin = from emp in Employee.GetAllEmployees() //Left Data Source
                             join add in Address.GetAddress() //Right Data Source
                             on emp.AddressId equals add.ID //Inner Join Condition
                             into EmployeeAddressGroup //Performing LINQ Group Join
                             from address in EmployeeAddressGroup.DefaultIfEmpty() //Left Outer Join
                             select new { emp, address }; //Projecting the Result to an
            //Performing Left Outer Join
            Anonymous Type
            //Accessing the Elements using For Each Loop
            foreach (var item in QSOuterJoin)
            {
                //Accessing the Elements
            }
        }
    }
}

```

```

        {
            //Before Accessing the AddressLine, please check null else you
            Null Reference Exception
            Console.WriteLine($"Name : {item.emp.Name}, Address :
            {item.address?.AddressLine} ");
        }
        Console.ReadLine();
    }
}

```

Now run the application, and you will get the following output. As you can see, even though Anurag and Pranaya do not have the corresponding address, they are still returned as part of the result set. In that case, the Address will be NULL.

```

Name : Preety, Address : AddressLine1
Name : Priyanka, Address : AddressLine2
Name : Anurag, Address :
Name : Pranaya, Address :
Name : Hina, Address : AddressLine5
Name : Sambit, Address : AddressLine6

```

Projecting the Result to a Named Type:

Instead of projecting the result to an anonymous type, we can project it to a named type. Let's see how. First, create a class file named **EmployeeResult.cs** with the required properties you want in the result set. As per our requirement, we have created the class with two properties.

```

namespace LINQLeftOuterJoin
{
    class EmployeeResult
    {
        public Employee Employee { get; set; }
        public Address Address { get; set; }
    }
}

```

Next, modify the Main method of the Program class as follows. Here, we are projecting the result to the above-created EmployeeResult type.

```

using System.Linq;
using System;
namespace LINQLeftOuterJoin
{
    class Program
    {
        static void Main(string[] args)
        {
            //Performing Left Outer Join using LINQ using Query Syntax

```

```

//Left Data Source: Employees
//Right Data Source: Addresses
//Note: Left and Right Data Source Matters
var QSOuterJoin = from emp in Employee.GetAllEmployees() //Left Data Source
                  join add in Address.GetAddress() //Right Data Source
                  on emp.AddressId equals add.ID //Inner Join Condition
                  into EmployeeAddressGroup //Performing LINQ GroupJoin
                  from address in EmployeeAddressGroup.DefaultIfEmpty() //Left Outer Join

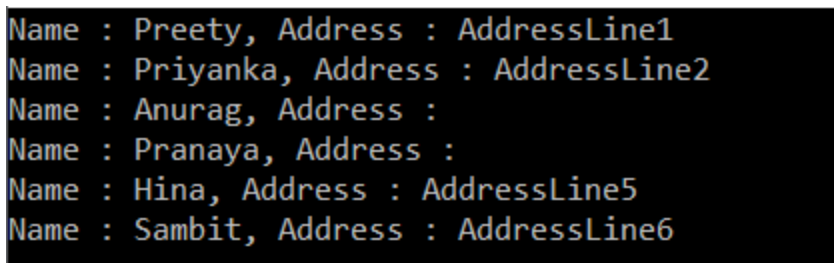
//Performing Left Outer Join

//Projecting the Result to Named Type
select new EmployeeResult
{
    Employee = emp,
    Address = address
};

//Accessing the Elements using For Each Loop
foreach (var item in QSOuterJoin)
{
    //Before Accessing the AddressLine, please check null else you will get
    Null Reference Exception
    Console.WriteLine($"Name : {item.Employee.Name}, Address : {item.Address?.AddressLine} ");
}
Console.ReadLine();
}
}
}

```

With the above changes in place, run the application, and you will also get the same output as the previous example, as shown in the below image.



```

Name : Preety, Address : AddressLine1
Name : Priyanka, Address : AddressLine2
Name : Anurag, Address : 
Name : Pranaya, Address : 
Name : Hina, Address : AddressLine5
Name : Sambit, Address : AddressLine6

```

Examples to Understand LINQ Left Outer Join using Method Syntax in C#:

To implement Left Outer Join in Linq using Method Syntax, we need to use the **GroupJoin()** method along with the **SelectMany()** and **DefaultIfEmpty()** methods. So, let us rewrite the previous example using the LINQ Method Syntax. For a better understanding, please have a look at the following example. The following example code is self-explained, so please go through the comment lines.

```

using System.Linq;
using System;
namespace LINQLeftOuterJoin

```

```

{
    class Program
    {
        static void Main(string[] args)
        {
            //Performing Left Outer Join using LINQ using Method Syntax
            //Left Data Source: Employees
            //Right Data Source: Addresses
            //Note: Left and Right Data Source Matters
            var MSOuterJOIN = Employee.GetAllEmployees() //Left Data Source
                //Performing Group join with Right Data Source
                .GroupJoin(
                    Address.GetAddress(), //Right Data Source
                    employee => employee.AddressId, //Outer Key
                    address => address.ID, //Inner Key Selector
                    (employee, address) => new { employee, address }
                )
                .SelectMany(
                    x => x.address.DefaultIfEmpty(), //Perform Left Outer Join
                    (employee, address) => new { employee, address }
                );
            //Final Result Set

            //Accessing the Elements using For Each Loop
            foreach (var item in MSOuterJOIN)
            {
                Console.WriteLine($"Name : {item.employee.employee.Name}, Address : {item.address?.AddressLine} ");
            }
            Console.ReadLine();
        }
    }
}

```

Now, run the application, which will give you the same output as the previous example, as shown in the image below. It is always better to use Query Syntax over Method Syntax to perform left outer join in Linq as it is simple and easy to understand.

```

Name : Preety, Address : AddressLine1
Name : Priyanka, Address : AddressLine2
Name : Anurag, Address : 
Name : Pranaya, Address : 
Name : Hina, Address : AddressLine5
Name : Sambit, Address : AddressLine6

```

Instead of projecting the result to an anonymous type, we can also project the result to a named type with LINQ Method Syntax. We have already created a type called EmployeeResult. Let us project the result to this

EmployeeResult type. So, modify the Main method of the Program class as follows to project the result to the EmployeeResult type.

```
using System.Linq;
using System;
namespace LINQLeftOuterJoin
{
    class Program
    {
        static void Main(string[] args)
        {
            //Performing Left Outer Join using LINQ using Method Syntax
            //Left Data Source: Employees
            //Right Data Source: Addresses
            //Note: Left and Right Data Source Matters
            var MSOuterJOIN = Employee.GetAllEmployees() //Left Data Source
                .GroupJoin(
                    Address.GetAddress(), //Right Data Source
                    employee => employee.AddressId, //Outer Key
                    address => address.ID, //Inner Key Selector
                    (employee, address) => new { employee, address }
                )
                .SelectMany(
                    x => x.address.DefaultIfEmpty(), //Perform
                    (employee, address) => new EmployeeResult
                    {
                        Employee = employee.employee,
                        Address = address
                    }
                );

            //Accessing the Elements using For Each Loop
            foreach (var item in MSOuterJOIN)
            {
                Console.WriteLine($"Name : {item.Employee.Name}, Address : {item.Address?.AddressLine} ");
                Console.ReadLine();
            }
        }
    }
}
```

Run the application, and you will get the same output as the previous example.

Anonymous Type with User-Defined Properties in the ResultSet:

Let us see how to return an anonymous type with user-defined properties using LINQ Left Join in C#. Please have a look at the following example for a better understanding. In the example below, we display the value NA if the corresponding address is unavailable.

```
using System.Linq;
using System;
namespace LINQLeftOuterJoin
{
    class Program
    {
        static void Main(string[] args)
        {
            //Using Method Syntax
            var MSOuterJOIN = Employee.GetAllEmployees()
                .GroupJoin(
                    Address.GetAddress(),
                    emp => emp.AddressId,
                    add => add.ID,
                    (emp, add) => new { emp, add }
                )
                .SelectMany(
                    x => x.add.DefaultIfEmpty(),
                    (employee, address) => new
                    {
                        EmployeeName = employee.emp.Name,
                        AddressLine = address == null ? "NA" :
address.AddressLine
                    }
                );

            //Using Query Syntax
            var QSOuterJoin = from emp in Employee.GetAllEmployees()
                              join add in Address.GetAddress()
                              on emp.AddressId equals add.ID
                              into EmployeeAddressGroup
                              from address in EmployeeAddressGroup.DefaultIfEr
                              select new
                              {
                                  EmployeeName = emp.Name,
                                  AddressLine = address == null ? "NA" :
address.AddressLine
                              };

            foreach (var item in MSOuterJOIN)
            {
                Console.WriteLine($"Name : {item.EmployeeName}, Address :
{item.AddressLine} ");
            }
            Console.ReadLine();
        }
    }
}
```

Now, run the application, and you will get the following output. Now, we are getting NA in the address if the corresponding address is unavailable for an employee.

```
Name : Preety, Address : AddressLine1
Name : Priyanka, Address : AddressLine2
Name : Anurag, Address : NA
Name : Pranaya, Address : NA
Name : Hina, Address : AddressLine5
Name : Sambit, Address : AddressLine6
```

LINQ Right Outer Join

The RIGHT OUTER JOIN retrieves all the matching rows from both the data sources involved in the join and non-matching rows from the right-side data source. In this case, the non-matching data will take the default values. However, Right Outer Joins are not supported with LINQ. LINQ only supports left outer joins.

Exchange the data sources to perform the right outer join. In our previous examples, we used Employees as the Left Data Source and Addresses as the Right Data Source. In the example below, we are just changing the data sources. Now, we are making addresses the Left Data Source and Employees the Right Data Source.

```
using System.Linq;
using System;
namespace LINQLeftOuterJoin
{
    class Program
    {
        static void Main(string[] args)
        {
            //Performing Right Outer Join using LINQ using Query Syntax
            //Changing the Data Sources
            //Left Data Source: Addresses
            //Right Data Source: Employees
            //Note: Left and Right Data Source Matters
            var QSRightJoin = from add in Address.GetAddress() //Left Data Source
                             join emp in Employee.GetAllEmployees() //Right Data Source
                             on add.ID equals emp.AddressId //Inner Join Condition
                             into EmployeeAddressGroup //Performing LINQ Group Join
                             from employee in EmployeeAddressGroup.DefaultIfEmpty()
            //Performing Left Outer Join
            select new { add, employee }; //Projecting the result into an Anonymous Type

            //Accessing the Elements using For Each Loop
            foreach (var item in QSRightJoin)
            {
                //Before Accessing the AddressLine, please check null else you will get
                //Null Reference Exception
            }
        }
    }
}
```

```

        Console.WriteLine($"Name : {item.employee?.Name}, Address :
{item.add?.AddressLine} ");
    }
    Console.ReadLine();
}
}
}

```

Real-Time Example to Understand LINQ Left Join:

Suppose you have a list of orders and a list of customers, and you want to generate a report of all orders along with the corresponding customer names (if available) or a default value if there's no matching customer. A left join would be appropriate in this case because you want to include all orders in the report. For a better understanding, please have a look at the below example:

```

using System;
using System.Collections.Generic;
using System.Linq;
namespace ConsoleApp12
{
    public class Order
    {
        public int OrderId { get; set; }
        public int CustomerId { get; set; }
        public DateTime OrderDate { get; set; }
    }
    public class Customer
    {
        public int CustomerId { get; set; }
        public string Name { get; set; }
    }
    class Program
    {
        static void Main(string[] args)
        {
            List<Order> orders = new List<Order>
            {
                new Order { OrderId = 101, CustomerId = 10101, OrderDate =
DateTime.Parse("2022-05-10") },
                new Order { OrderId = 102, CustomerId = 10102, OrderDate =
DateTime.Parse("2022-05-15") },
                new Order { OrderId = 103, CustomerId = 10103, OrderDate =
DateTime.Parse("2022-05-20") },
            };
            List<Customer> customers = new List<Customer>
            {
                new Customer { CustomerId = 10101, Name = "Pranaya" },
                new Customer { CustomerId = 10103, Name = "Rout" },
            };
            var leftJoinQuery =

```

```

        from order in orders
        join customer in customers
        on order.CustomerId equals customer.CustomerId into customerGr
        from customerInfo in customerGroup.DefaultIfEmpty()
        select new
        {
            OrderId = order.OrderId,
            OrderDate = order.OrderDate,
            CustomerName = customerInfo?.Name ?? "No Customer"
        };
        foreach (var result in leftJoinQuery)
        {
            Console.WriteLine($"OrderID: {result.OrderId}, OrderDate:
{result.OrderDate}, CustomerName: {result.CustomerName}");
        }
        Console.ReadKey();
    }
}
}

```

In the above example, we first define two collections: orders and customers. We then use a LINQ query to perform a left join between these collections based on the CustomerId field. The into keyword creates a temporary grouping of results for each order, and the DefaultIfEmpty method specifies the left join behavior.

The result is a sequence of anonymous objects that contain order information along with customer names (if available) or “No Customer” if no matching customer is found for an order.

Another Real-Time Example to Understand LINQ Left Join

To understand LINQ left join with a real-time example, let’s consider another scenario where we have two collections: Employees and Departments. Each employee belongs to a department, but there might be departments without employees. We want to list all departments along with their employees, including those with no employees. This is a typical use case for a left join.

Here’s how the scenario looks:

- **Departments Collection: DepartmentID, DepartmentName**
- **Employees Collection: EmployeeID, EmployeeName, DepartmentID**

We want to join these collections to list each department and its employees. Departments without employees should also be listed (with null or an appropriate placeholder for the EmployeeName). In C#, using LINQ, the query would look something like this:

```

using System;
using System.Collections.Generic;
using System.Linq;
namespace LINQLeftOuterJoin
{
    public class Program
    {

```

```

public static void Main()
{
    // Sample Departments
    List<Department> departments = new List<Department>()
    {
        new Department{ DepartmentID = 1, DepartmentName = "IT"},
        new Department{ DepartmentID = 2, DepartmentName = "HR"},
        new Department{ DepartmentID = 3, DepartmentName = "Finance"}
    };
    // Sample Employees
    List<Employee> employees = new List<Employee>()
    {
        new Employee{ EmployeeID = 1, EmployeeName = "John Doe", Depart
1},
        new Employee{ EmployeeID = 2, EmployeeName = "Jane Doe", Depart
1},
        new Employee{ EmployeeID = 3, EmployeeName = "Jim Beam", Depart
2}
    };
    var departmentEmployees = from department in departments
                              join employee in employees
                              on department.DepartmentID equals
employee.DepartmentID into deptEmps
                              from employee in deptEmps.DefaultIfEmpty
                              select new
                              {
                                  DepartmentName = department.DepartmentName,
                                  EmployeeName = employee?.EmployeeName
Employees"
                              };
    foreach (var deptEmp in departmentEmployees)
    {
        Console.WriteLine($"Department: {deptEmp.DepartmentName}, EmpID: {deptEmp.EmployeeID}, EmployeeName: {deptEmp.EmployeeName}");
    }
}

public class Department
{
    public int DepartmentID { get; set; }
    public string DepartmentName { get; set; }
}

public class Employee
{
    public int EmployeeID { get; set; }
    public string EmployeeName { get; set; }
    public int DepartmentID { get; set; }
}
}

```

Output:

```
Department: IT, Employee: John Doe  
Department: IT, Employee: Jane Doe  
Department: HR, Employee: Jim Beam  
Department: Finance, Employee: No Employees
```

When Should We Use LINQ Left Join in C#?

Here are some common scenarios where you might use a LINQ left join:

- **Combining Data with Missing Records:** When you have two collections, and you want to combine them while preserving all elements from one collection and including data from the other collection when a match is found, but not excluding any elements from the first collection.
- **Optional Relationships:** In database scenarios, if you have a parent-child relationship between two tables where some child records may not have a corresponding parent record, a left join ensures that you get all child records along with their parent data when available.
- **Handling Missing Data:** When dealing with incomplete datasets, you need to ensure that the output includes all available data from one dataset, even if it can't be fully matched with the other dataset.
- **Reporting and Analysis:** For reporting purposes, you need to produce comprehensive lists that include all items from one dataset with corresponding details from another dataset when available. For example, you could generate a report of all sales transactions and include related customer information, even for transactions without linked customer data.
- **Handling Default Values:** When combining data, you can use left joins to provide default values or placeholders for missing data from the second collection.

In the next article, I will discuss the [LINQ Full Outer Join in C#](#) with Examples. In this article, I explain how to implement **Left Join in Linq** using Method syntax and Query Syntax using C# Language. I hope you enjoy this LINQ Left Outer Join in C# with Examples article.